Università Commerciale Luigi Bocconi

20532 Macroeconometrics

Problem Set 1

Explanatory Note

Author: Alex Vigna Lobbia, Giorgio Ucropina and Marco Calia
Course: 20532 Macroeconometrics

Contents

1 Introduction and AI disclosure **2**

2 Question 1: Simulation of an AR(1) Process **2**

3 Question 2: Initial Conditions and Burn-In **3**

4 Question 3: Monte Carlo Distribution of the OLS Estimator **5**

5 Question 4: Effect of Sample Size **6**

6 Question 5: Fitting an AR(1) to MA(1) Data **8**

7 Question 6: General ARMA(p, q) Generator **11**

Introduction and AI disclosure

This explanatory note presents and discusses the numerical exercises contained in Problem Set 1. The analysis focuses on simulation of stationary time–series processes, finite–sample properties of OLS estimators, Monte Carlo experiments, and the construction of a general ARMA(p, q) simulator.

The MATLAB implementation is contained in `PS1_complete_solution.m`, with the auxiliary ARMA function implemented in `arma_generator_ps1.m`.

Finally, the AI disclosure: we used Codex and ChatGPT (Sol High Model) to i) polish both text and outline of the explanatory note (to have a beautiful file without typos!); ii) improve portability of the Matlab code while also allowing for a polished code.

Question 1: Simulation of an AR(1) Process

> *Generate 500 observations from an AR(1) process Y_t with $\mathbb{E}[Y_t] = 0$, $\phi = 0.4$, and variance of the white-noise forcing term $\sigma_\varepsilon^2 = 0.2$ using the two methods below (hint: use a seed when you start the simulation).*
>
> a. *A `for` loop using the recursive structure of the AR(1).*
> b. *Using the function `filter`.*
> c. *Check that when the forcing variables are the same, the output of the two approaches is the same (be careful with the starting conditions and with the random number generator).*

We simulate the stationary AR(1) process

$$y_t = \phi y_{t-1} + \varepsilon_t, \qquad \phi = 0.4, \qquad \varepsilon_t \overset{\text{iid}}{\sim} \mathcal{N}(0, 0.2),$$

so that the unconditional mean is zero. We set the initial condition to $y_0 = 0$ and fix the random-number generator with `rng(101,'twister')`. MATLAB's `randn` generates standard normal draws, which are multiplied by $\sqrt{0.2}$ to obtain innovations with the required variance. Importantly, the same innovation vector is used in both simulation methods.

For the first method, the process is generated recursively using a `for` loop:

$$y_t = 0.4 y_{t-1} + \varepsilon_t.$$

The vector containing the simulated observations is preallocated with `zeros`, and at every iteration the newly generated value becomes the lagged value used in the following period.

For the second method, we write the same process in lag-polynomial form,

$$(1 - 0.4L) y_t = \varepsilon_t,$$

and generate it using `filter(1,[1 -phi1],epsilon1)`. Since MATLAB's `filter` starts from a zero initial state, choosing $y_0 = 0$ in the loop makes the starting conditions identical across the two implementations.

Figure 1 compares the two simulated paths. The paths overlap observation by observation, while the lower panel plots their period-by-period difference. The maximum absolute difference and the root mean squared difference are zero up to numerical precision. Therefore, once both

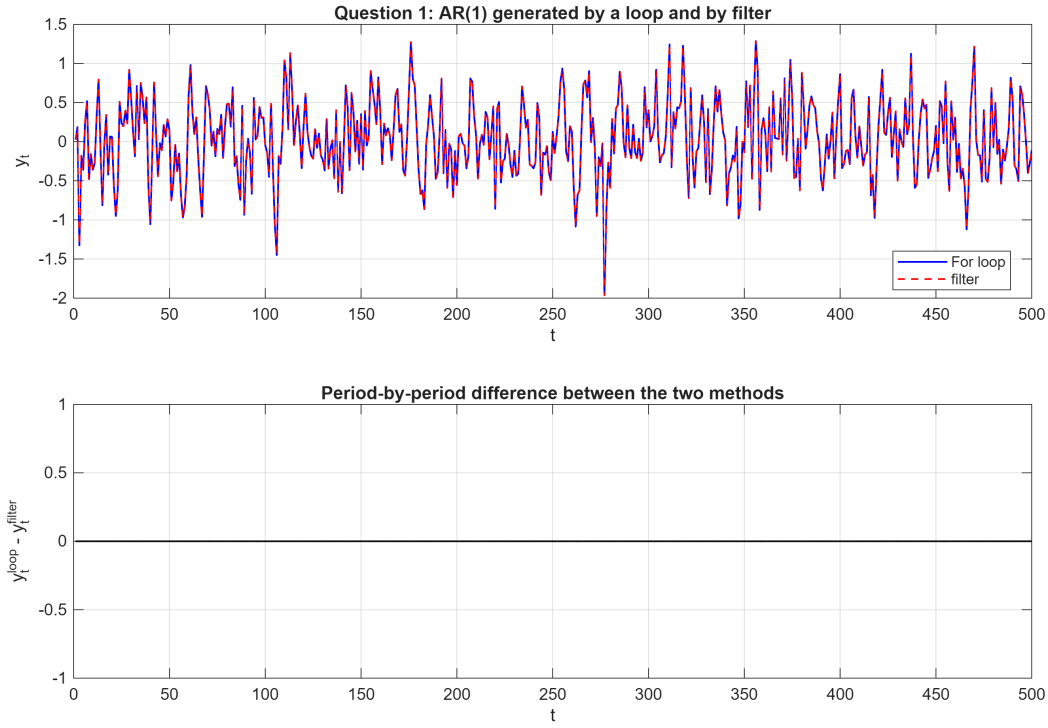


Figure 1: AR(1) realization generated using the explicit recursion and MATLAB's `filter`. The lower panel reports their observation-by-observation difference.

the forcing variables and the initial condition are held fixed, the recursive implementation and `filter` are simply two numerical representations of the same AR(1) recursion.

In a nutshell, Q.1 verifies that an AR(1) process can be simulated equivalently either through its recursive definition using a for loop or through its lag–polynomial representation using MATLAB's filter function. As we showed, when both methods use the same innovation sequence and the same initial condition, they produce the same realization up to numerical precision.

Question 2: Initial Conditions and Burn-In

> *Generate data from an AR(1) with* $\phi = 0.6$, $\sigma_\varepsilon^2 = 0.4$, *and* $\mathbb{E}[Y_t] = 3$. *Set the starting condition of your simulation to* 20. *What happens if the starting condition you choose is far from the unconditional mean of the process? What would you do in order to make sure that the sample path is a "proper" realization of the stationary process you want to simulate from? You can use either* `for` *or* `filter`.

The process in Question 2 has

$$\phi = 0.6, \qquad \sigma_\varepsilon^2 = 0.4, \qquad \mathbb{E}[y_t] = \mu = 3.$$

Writing the AR(1) as

$$y_t = c + \phi y_{t-1} + \varepsilon_t,$$

the intercept consistent with the required unconditional mean is

$$c = \mu(1 - \phi) = 3(1 - 0.6) = 1.2.$$

The simulation is deliberately initialized at $y_0 = 20$, far above the unconditional mean.

Conditional on this initial value, the conditional expectation of y_t given the initial value y_0, i.e. the optimal forecast based on the information available at the starting date (in the notes, obtained by iterating the AR(1) forward and taking conditional expectations)

$$\mathbb{E}[y_t \mid y_0] = \mu + \phi^t(y_0 - \mu) = 3 + 0.6^t(20 - 3).$$

Hence the effect of the initial condition decreases geometrically as t increases because $|\phi| < 1$. The first panel of Figure 2 illustrates this transient: the process initially falls rapidly from 20 towards its unconditional mean, while random innovations generate fluctuations around the conditional expected path. For a stationary AR(1), the forecast progressively converges to the unconditional mean as the forecast horizon increases.

A sample beginning directly from $y_0 = 20$ is therefore not immediately representative of the stationary distribution. To obtain a proper stationary realization, we simulate 500 additional observations and treat them as a burn-in period. The complete recursion is run for $T + 500$ periods and the first 500 observations are discarded.

The second panel of Figure 2 shows the retained 500 observations after burn-in. The series now fluctuates around the unconditional mean of 3 without any economically relevant dependence on the arbitrary starting value. Thus the role of the burn-in is not to remove the stochastic variation of the process, but to remove the transient effect generated by the initialization.

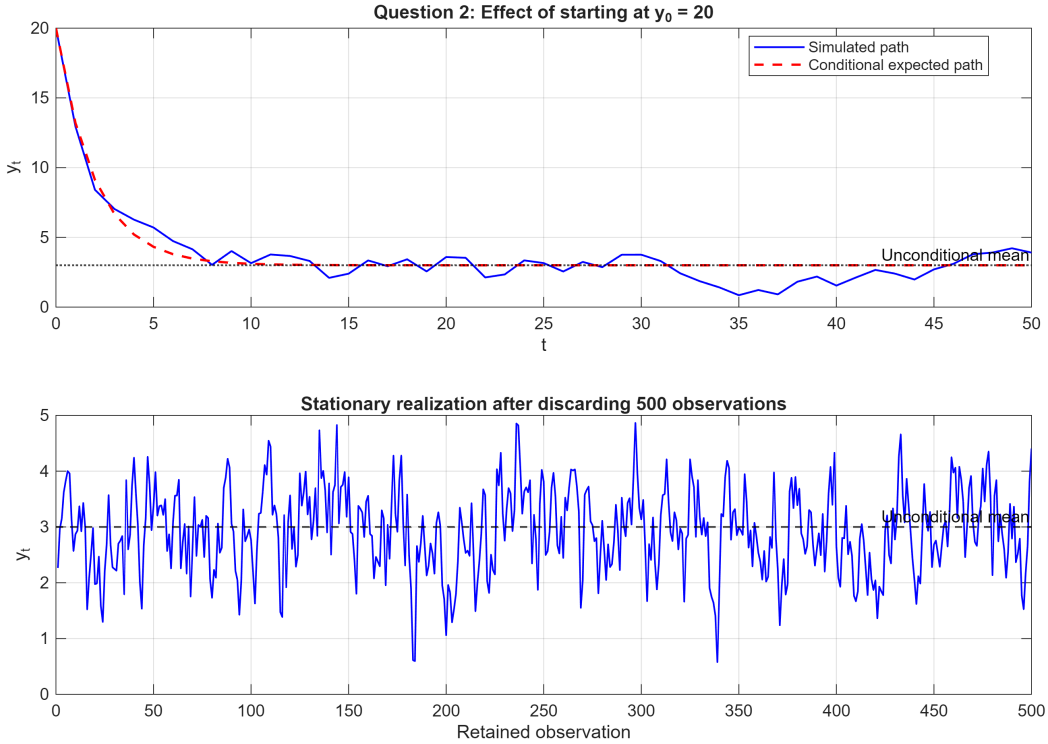


Figure 2: Effect of the distant initial condition and the retained realization after a 500-observation burn-in. The upper panel compares the simulated path with its conditional expected path; the lower panel shows the post-burn-in sample around the unconditional mean.

In a nutshell, Q.2 shows that when the starting condition of a stationary AR(1) process is far from its unconditional mean, the effect of that initial value gradually disappears over time, provided that $|\phi| < 1$. As the process converges back towards its unconditional mean, a convenient way to obtain a "proper" realization of the stationary process is to simulate additional initial observations and discard them as a burn-in period.

Question 3: Monte Carlo Distribution of the OLS Estimator

Compute the empirical distribution of the OLS estimator in the case of an AR(1) with $\phi = 0.4$ and $T = 250$ (you are free to choose the variance of the innovation). Construct a t-test for the null hypothesis $H_0 : \phi = 0$ against the two-sided alternative $H_1 : \phi \neq 0$. How often do you reject H_0 at the 95% confidence level when $T = 250$?

We simulate the AR(1) process

$$y_t = 0.4y_{t-1} + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, 1).$$

Thus the chosen innovation variance is one and the unconditional mean is zero. We use $R = 5{,}000$ replications, retaining $T = 250$ observations in each. In every replication, `randn` generates 250+300 innovations and `filter(1,[1 -phi3],epsilon3)` applies the AR recursion from a zero initial state. We discard the first 300 observations, making the effect of initialization negligible. The command `rng(303,'twister')` fixes the seed once before the simulation loop; each replication then uses fresh innovations.

Although the true intercept is zero, the estimated regression includes a constant, as in the supplied code. Relabelling the retained observations as $y_1, \ldots, y_T$, we construct

$$Y = \begin{pmatrix} y_2 \\ \vdots \\ y_T \end{pmatrix}, \qquad X = \begin{pmatrix} 1 & y_1 \\ \vdots & \vdots \\ 1 & y_{T-1} \end{pmatrix}, \qquad \widehat{\beta} = \begin{pmatrix} \widehat{c} \\ \widehat{\phi} \end{pmatrix} = (X'X)^{-1}X'Y.$$

There are $n = T - 1 = 249$ regression observations. MATLAB computes the coefficients with `X3\Y3`; we store the slope estimate from each replication to obtain its empirical sampling distribution.

Construction of the two-sided test

For each replication, we test $H_0 : \phi = 0$ against $H_1 : \phi \neq 0$. Writing $\widehat{u} = Y - X\widehat{\beta}$, the supplied code calculates

$$\widehat{\sigma}^2 = \texttt{cov}(\widehat{u}) = \frac{\sum_{i=1}^{n}(\widehat{u}_i - \overline{\widehat{u}})^2}{n-1}, \qquad \widehat{\mathrm{Var}}(\widehat{\beta}) = \widehat{\sigma}^2(X'X)^{-1},$$

$$\widehat{\mathrm{se}}(\widehat{\phi}) = \sqrt{\left[\widehat{\mathrm{Var}}(\widehat{\beta})\right]_{22}}, \qquad t = \frac{\widehat{\phi}}{\widehat{\mathrm{se}}(\widehat{\phi})}.$$

These calculations use `cov`, `inv`, and `sqrt`. The intercept makes the residual mean zero up to numerical precision, so this variance estimate is SSE/248. This records the code's convention: the usual correction for two estimated regression coefficients would instead give SSE/247.

The rejection rule is $|t| > 1.96$, corresponding to a two-sided test at the nominal 5% significance level (the 95% confidence level). The cutoff is a large-sample standard-normal approximation, not an exact finite-sample Student-t critical value. The empirical rejection frequency is

$$\widehat{p}_{\text{reject}} = \frac{1}{R}\sum_{r=1}^{R} \mathbf{1}\{|t_r| > 1.96\}.$$

Empirical results and interpretation

The saved draws give the following summary. The standard deviation is computed across replications, and the bias is the mean minus 0.4.

Statistic	Result
True coefficient ϕ	0.400000
Monte Carlo mean of $\widehat{\phi}$	0.389299
Monte Carlo standard deviation	0.059005
Monte Carlo bias	−0.010701
Rejections / replications	5,000 / 5,000
Rejection frequency	100%

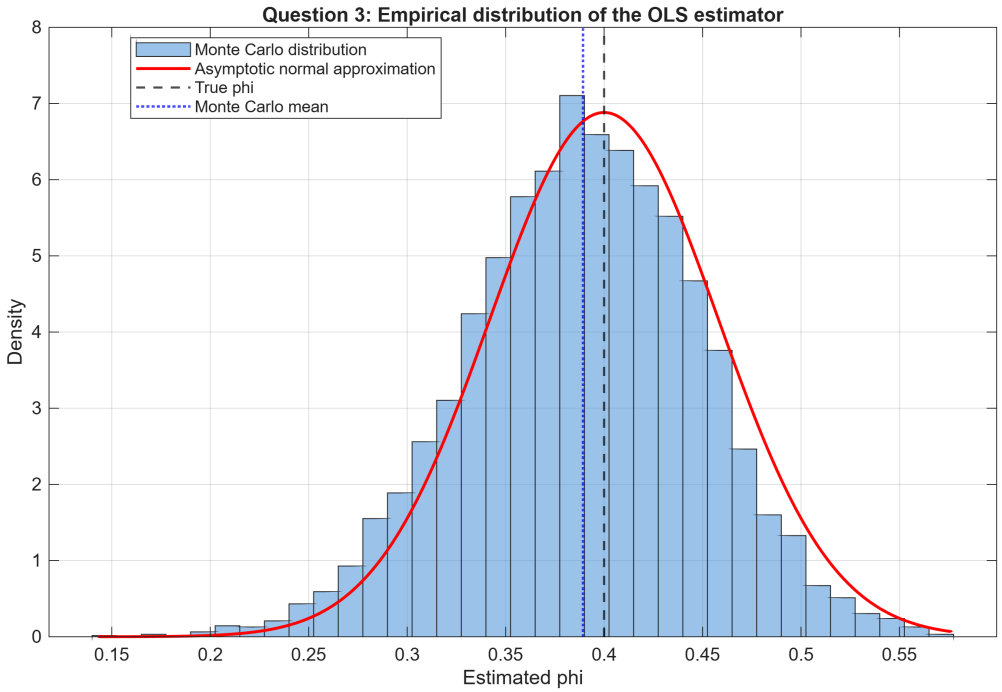


Figure 3: Empirical distribution of $\widehat{\phi}$ for $\phi = 0.4$ and $T = 250$, based on 5,000 replications. The histogram is normalized as a density. The dashed black line marks the true coefficient, the dotted blue line the Monte Carlo mean, and the red curve the large-sample normal approximation.

Figure 3 is approximately bell-shaped, with a small downward shift relative to 0.4. For a stationary AR(1), the large-sample benchmark is

$$\sqrt{T}(\widehat{\phi} - \phi) \xrightarrow{d} \mathcal{N}(0, 1 - \phi^2).$$

Accordingly, the plotted approximation is $\mathcal{N}(0.4, 0.84/250)$, whose standard deviation is 0.057966, close to the empirical 0.059005. The script uses T in this benchmark; using the actual $T - 1$ regression pairs is asymptotically equivalent. The approximation describes the dispersion well, but does not capture the observed finite-sample bias.

We reject H_0 in all 5,000 replications. Since the data are generated with $\phi = 0.4$, this frequency estimates the test's *power* against that alternative, rather than its size under the null. The experiment therefore indicates very high power at $T = 250$; a finite simulation with no non-rejections does not establish that population power is exactly one.

Question 4: Effect of Sample Size

Compute the empirical distribution of the OLS estimator in the case of an AR(1) with $\phi = 0.9$ and $T \in \{50, 100, 200, 1000\}$. For each T, do 1000 simulations and

plot the distribution. How is the distribution changing with T?

We now simulate

$$y_t = 0.9y_{t-1} + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0,1),$$

for $T \in \{50, 100, 200, 1000\}$. For each sample size, we perform exactly 1,000 replications. We use `randn` and `filter` as above, discarding 500 initial observations from each path. This longer burn-in makes the initialization effect negligible even for the more persistent process with $\phi = 0.9$. The seed is set once with `rng(404,'twister')` before the loops over sample sizes and replications.

For every retained sample, we regress y_t on a constant and y_{t-1} using `X4\Y4`, giving $T-1$ regression pairs. Thus the estimation procedure is the same as in point 3. The Monte Carlo means, standard deviations, and biases are reported in Table 1.

Table 1: Sampling distribution of $\widehat{\phi}$ when $\phi = 0.9$. Each row uses 1,000 replications and 500 burn-in observations.

T	Mean	Empirical SD	Bias	Asymptotic SD
50	0.815317	0.095729	−0.084683	0.061644
100	0.861133	0.053508	−0.038867	0.043589
200	0.880888	0.036055	−0.019112	0.030822
1000	0.896635	0.014449	−0.003365	0.013784

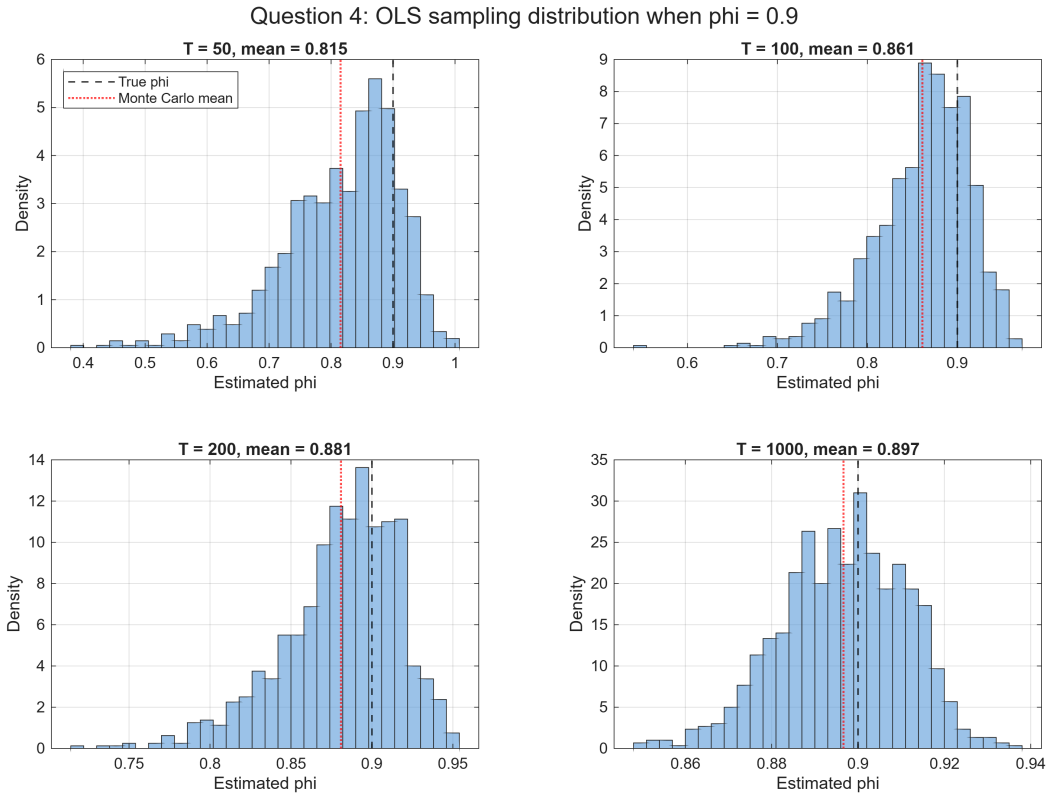


Figure 4: Empirical OLS distributions for the four sample sizes. Each density-normalized histogram contains 1,000 estimates. Dashed black lines mark $\phi = 0.9$ and dotted red lines mark the Monte Carlo means. The panels use different axis scales, so the numerical standard deviations in Table 1 are useful for comparing dispersion.

How the distribution changes with sample size

The downward bias decreases. At $T = 50$, the mean estimate is 0.815317, well below 0.9, with bias -0.084683. The means move towards the true value as T increases, reaching 0.896635 at $T = 1000$, with bias -0.003365. This is consistent with the finite-sample downward bias of OLS in an autoregression with an estimated intercept. Burn-in makes the influence of the arbitrary starting value negligible; it does not eliminate the estimation bias arising in a short sample.

The distribution becomes more concentrated. The empirical standard deviation falls from 0.095729 at $T = 50$ to 0.014449 at $T = 1000$. Longer samples therefore deliver more precise estimates, and the distribution concentrates closer to the true coefficient. These results are consistent with consistency of OLS for this stationary AR(1).

The shape becomes closer to a normal distribution. In Figure 4, the short-sample histograms have a pronounced left tail. At $T = 1000$ the distribution is more symmetric and approximately bell-shaped, although some asymmetry remains. Since $\phi = 0.9 < 1$ is fixed, the stationary large-sample approximation applies:

$$\widehat{\phi} \overset{a}{\sim} \mathcal{N}\left(0.9, \frac{0.19}{T}\right), \qquad \mathrm{SD}_{\mathrm{asymp}}(\widehat{\phi}) = \sqrt{\frac{0.19}{T}}.$$

The last column of Table 1 reports this benchmark. At $T = 50$, the asymptotic standard deviation 0.061644 substantially understates the observed 0.095729. At $T = 1000$, 0.013784 is close to the observed 0.014449. Together with the reduction in bias and asymmetry, this shows why the normal approximation is more useful for larger samples in this experiment.

In MATLAB, `mean` and `std` summarize each set of slope estimates. The functions `histogram`, `subplot`, and `xline` generate the four panels and their reference lines. The results are saved in `question4_summary.csv` and `question4_monte_carlo_draws.csv`; the figure is the original `question4_distributions_by_sample_size.png` output.

Question 5: Fitting an AR(1) to MA(1) Data

> *Compute the empirical distribution (do 1000 simulations) of the OLS estimator in the regression*
>
> $$x_t = ax_{t-1} + v_t$$
>
> *in the case in which the data-generating process for x_t is MA(1) with $\theta = 0.6$ for $T \in \{50, 100, 200, 1000\}$. What is the mean of the distributions? Does the mean converge to anything as T increases? Discuss.*

We now generate data from the MA(1) process

$$x_t = \varepsilon_t + \theta\varepsilon_{t-1}, \qquad \theta = 0.6, \qquad \varepsilon_t \overset{\text{iid}}{\sim} \mathcal{N}(0, 1),$$

but estimate the AR(1) regression stated in the question,

$$x_t = ax_{t-1} + v_t,$$

without an intercept. For each $T \in \{50, 100, 200, 1000\}$, we perform 1,000 Monte Carlo replications. The seed is fixed once, before the simulation loops, with `rng(505,'twister')`.

In each replication, the code generates $T + 1$ innovations. The additional draw supplies the presample innovation ε_0, allowing all T observations to be constructed directly from the

stationary MA(1) representation. The regression then uses the $T-1$ pairs (x_t, x_{t-1}) and computes

$$\widehat{a} = \frac{\sum_{t=2}^{T} x_{t-1}x_t}{\sum_{t=2}^{T} x_{t-1}^2}.$$

Population projection coefficient

Although the data-generating process contains no autoregressive term, x_t and x_{t-1} are correlated because they share the innovation ε_{t-1}. In particular,

$$\gamma_0 = \mathrm{Var}(x_t) = (1+\theta^2)\sigma_\varepsilon^2,$$
$$\gamma_1 = \mathrm{Cov}(x_t, x_{t-1}) = \theta\sigma_\varepsilon^2.$$

Consequently, OLS converges to the population linear-projection coefficient

$$a^* = \operatorname{plim} \widehat{a} = \frac{\gamma_1}{\gamma_0} = \frac{\theta}{1+\theta^2}.$$

For $\theta = 0.6$,

$$a^* = \frac{0.6}{1+0.6^2} = 0.441176.$$

Thus the probability limit is not zero: it equals the lag-one autocorrelation of the MA(1) process.

Monte Carlo results

Table 2 reports the mean and standard deviation of the 1,000 estimates for each sample size. The fourth column is the Monte Carlo mean minus the theoretical limit a^*.

Table 2: Empirical distribution of the OLS estimator for MA(1) data. Each row is based on 1,000 replications.

T	Mean	Empirical SD	Mean minus a^*	a^*
50	0.429578	0.111080	−0.011598	0.441176
100	0.439028	0.075607	−0.002148	0.441176
200	0.435737	0.055125	−0.005439	0.441176
1000	0.440819	0.023007	−0.000358	0.441176

Figure 5 shows that the empirical distributions become progressively more concentrated around a^* as the sample size increases. The empirical standard deviation falls from 0.111080 at $T = 50$ to 0.023007 at $T = 1000$. The Monte Carlo mean at $T = 1000$ is 0.440819, which is very close to the theoretical value 0.441176.

The movement of the simulated means need not be monotonic. In particular, the mean at $T = 200$ is slightly farther from a^* than the mean at $T = 100$. This small reversal is compatible with Monte Carlo variation because each mean is itself estimated from a finite number of replications. The relevant pattern is that the sampling distribution contracts and its center approaches a^* as T becomes large. The experiment therefore confirms that fitting an AR(1) regression to MA(1) data consistently estimates the pseudo-true linear-projection coefficient rather than a structural autoregressive parameter.

In MATLAB, `randn` generates the innovations, the backslash operator `X5\Y5` computes the no-intercept OLS estimate, and `mean` and `std` summarize its Monte Carlo distribution. The functions `histogram`, `subplot`, `xline`, and `yline` produce Figures 5 and 6. The numerical results are saved in `question5_summary.csv` and `question5_monte_carlo_draws.csv`.

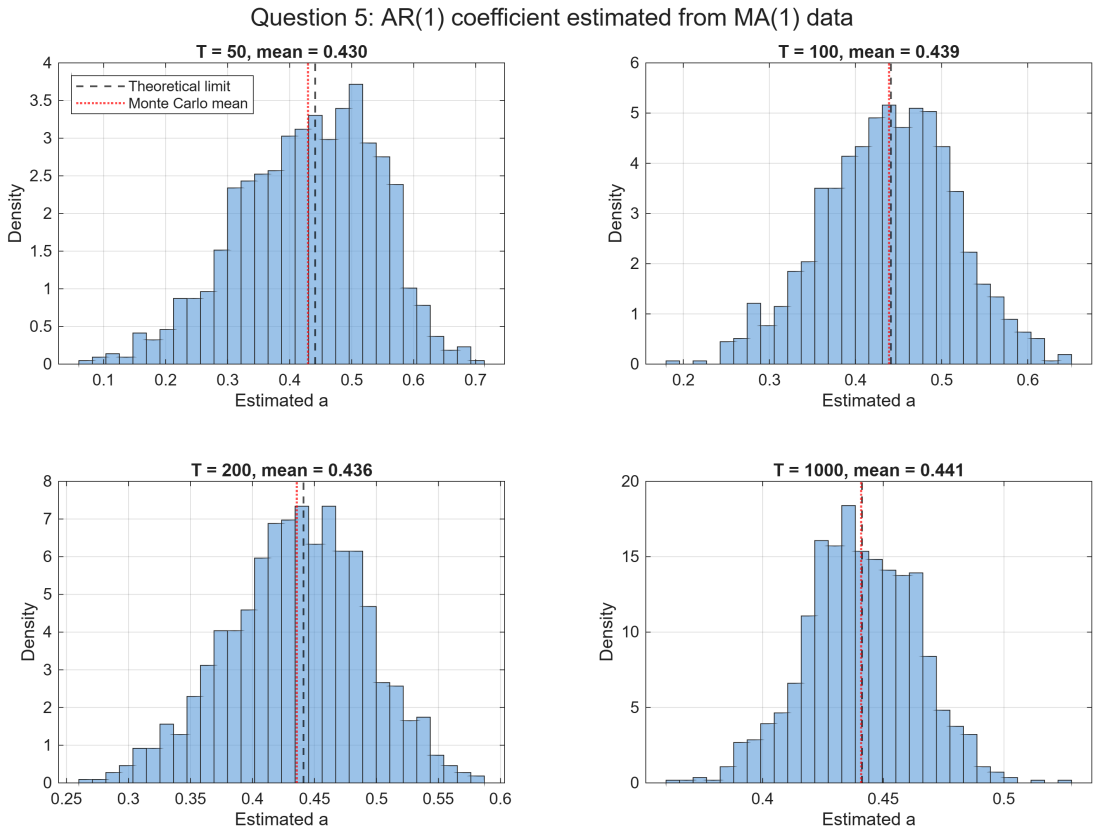


Figure 5: Empirical distributions of $\widehat{a}$ for the four sample sizes. Each density-normalized histogram contains 1,000 estimates. Dashed black lines mark the theoretical limit $a^* = 0.441176$, while dotted red lines mark the corresponding Monte Carlo means. The panels use different horizontal scales, so dispersion is best compared using Table 2.

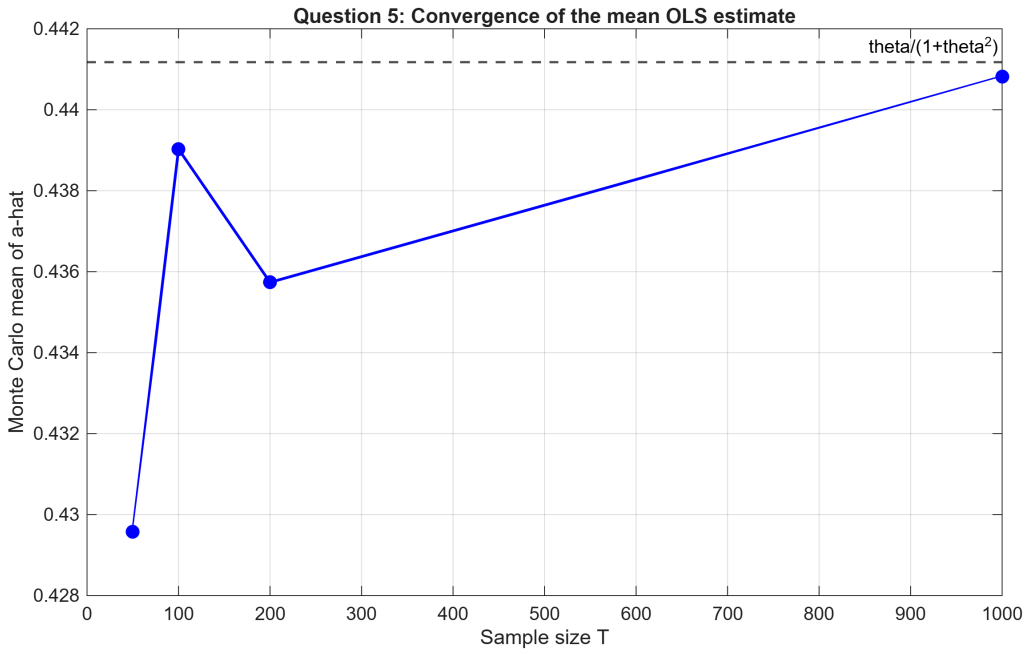


Figure 6: Monte Carlo mean of $\widehat{a}$ at each sample size. The dashed horizontal line is the population projection coefficient $a^* = \theta/(1+\theta^2)$.

Question 6: General ARMA(p, q) Generator

Write a function that generates T observations from an ARMA(p, q) using the `for`*-loop approach. The function must have the following inputs:*

1) *the number of observations;*
2) *the variance of the white noise ε_t;*
3) *the coefficients of the AR and MA polynomials or the roots of the AR and MA polynomials (hint: you may find the* `poly` *and* `roots` *functions useful).*

As output, the function must return the realizations of the ARMA(p, q) and the white noise ε_t.

We implement a general function for simulating an ARMA(p, q) process using explicit loops. In lag-polynomial notation,

$$A(L)y_t = B(L)\varepsilon_t,$$

where

$$A(L) = 1 + a_1 L + \cdots + a_p L^p, \qquad B(L) = 1 + b_1 L + \cdots + b_q L^q, \qquad \varepsilon_t \overset{\text{iid}}{\sim} \mathcal{N}(0, \sigma_\varepsilon^2).$$

Solving the equation for the current observation gives the recursion used by the function:

$$y_t = -\sum_{i=1}^{p} a_i y_{t-i} + \varepsilon_t + \sum_{j=1}^{q} b_j \varepsilon_{t-j}.$$

Presample values of both y_t and ε_t are initialized to zero. The function first simulates the requested burn-in observations, discards them, and returns the following T observations together with their innovations. The arguments can be supplied either as polynomial coefficients or as roots; in the latter case MATLAB's `poly` reconstructs the coefficient vectors.

ARMA(1,2) example

We test the function using $T = 500$, a burn-in of 500 observations, and $\sigma_\varepsilon^2 = 0.4$. The chosen polynomials are

$$A(L) = 1 - 0.8L, \qquad B(L) = 1 + 0.4L + 0.2L^2,$$

so the simulated process satisfies

$$y_t = 0.8y_{t-1} + \varepsilon_t + 0.4\varepsilon_{t-1} + 0.2\varepsilon_{t-2}.$$

The seed is reset to 606 before each call. Therefore the coefficient and root parameterizations receive exactly the same innovation sequence and can be compared observation by observation.

Here "roots" follows MATLAB's coefficient convention: applying `roots` to a coefficient vector and then `poly` to the result recovers that vector. This convention is useful for verifying the two input modes; it should not be confused with interpreting the zeros of a lag polynomial directly in powers of L.

Numerical validation

The first two checks compare the function's coefficient and root input modes. We also perform an independent check against MATLAB's `filter`. For this second comparison the burn-in

is set to zero so that both methods use the same zero initial conditions, and seed 707 makes their innovations identical. The benchmark is

$$y^{\text{filter}} = \texttt{filter}(B, A, \varepsilon).$$

Table 3: Observation-by-observation validation of the ARMA generator.

Check	Maximum absolute difference
Innovations, coefficient versus root input	0
Series, coefficient versus root input	8.8818×10^{-16}
Innovations, loop versus `filter`	0
Series, loop versus `filter`	8.8818×10^{-16}

The two nonzero differences are at machine-precision scale and arise only from floating-point arithmetic. Economically and numerically, the generated series are identical. The zero innovation differences also confirm that the comparisons use the same random shocks rather than merely two draws from the same distribution.

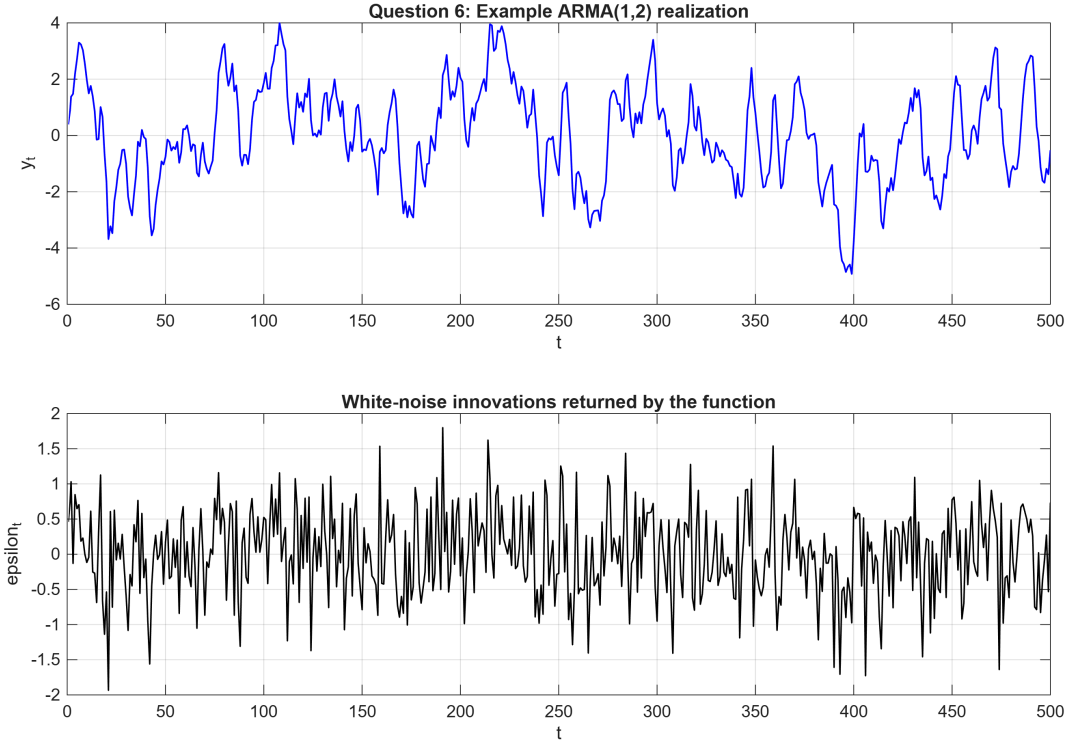


Figure 7: The retained ARMA(1,2) realization and the corresponding innovations returned by the general loop-based function. The persistent movements in the upper panel are consistent with the positive AR coefficient, while the lower panel displays the white-noise shocks.

Figure 7 provides a visual check, while Table 3 is the decisive numerical verification. The function therefore correctly handles the AR and MA recursions, burn-in, returned innovations, and both supported parameterizations for this test case. The series and validation statistics are saved in `question6_arma_example.csv` and `question6_summary.csv`.